Team 240

AI Incident Report Assistant: M2 Progress Report

Megathon X · Qualcomm Problem Statement 1 (On-Device AI Challenge) · M2: Mid-submission

Summary. The product is unchanged since M1: an employee speaks a report, the system identifies the incident type and department, loads that type's own form, fills it from the narration, asks a few bounded questions, and passes an explicit REVIEW before SUBMIT, all on-device. **Our end-to-end system now works: a back-and-forth conversation is feasible, and the system has successfully taken a report through to a submitted incident.** Since M1 we (1) ported the routing, review, submit and admin logic, first built and tested in Python, into the native Android/Kotlin app, now the single deliverable (17 JVM and 57 on-device tests, 0 failures), and (2) benchmarked the incident classifier on the Snapdragon kit: fine-tuned Laya reaches 87.6% type accuracy at 10.9 ms per turn on the NPU. Open items are the live map, a spoken test set and full-loop latency measurement.

I. Current Approach and Changes Since M1

The product, unchanged. An employee says, unprompted, "there's water pooling near the pantry, I think a pipe burst." There is no menu, category picker or visible form. While they talk, a lightweight always-on classifier has already proposed an incident type (Water Leak), a department (Facilities) and a location ("pantry") before any question is asked.

- **Confident case.** If the top candidate scores $\geq$0.8 and is at least 0.15 clear of the next, the app asks only what is still missing for *that* type's form: a Water Leak form asks about source and severity; an access-control fault asks which door and whether anyone is locked out. This is the core idea: do not make everyone fill one generic form. Forms are loaded from local admin configuration, not hard-coded per type.
- **"I don't know"** is accepted for any required field, stored as a real answer, never silently re-asked and never invented. A few optional clarifying questions follow (severity detail, whether anyone is in danger), stopped by relevance and a hard 6-turn bound, not a fixed script.
- **REVIEW.** The employee sees and hears the full report and gets four choices: confirm; correct one detail (which invalidates anything derived from the old value, so a changed description resets the stored priority to be re-evaluated, not left stale); skip remaining clarifications; or cancel with nothing saved. Only on confirm does an incident exist: it lands in the Facilities queue and a pin appears on the admin's floor-plan map at the zone matching "pantry."
- **Fully on-device.** Classifier, semantic model with function calling, speech recognition, text-to-speech, map and admin console run on the device. Nothing calls the internet at run time; this is a hard constraint of the problem statement, shown live by disconnecting the kit's network before the demo.

Change 1: target runtime. At M1 the plan had two kit-bound phases (model benchmarking, full on-device pipeline integration) in parallel with two kit-free phases (routing and form engine; review, submit, admin, versioning), joined by a frozen interface contract and merged at the end. The kit-free half was built that way: in Python, against SQLite, against a scripted stub standing in for the real models, and tested in isolation. When the kit-bound work was reviewed, it was a complete native Android application in Kotlin, with JNI bridges into whisper.cpp and llama.cpp, not a Python service that could sit beside ours. Keeping two runtimes talking over localhost, on hardware nobody had finished benchmarking, under a hard deadline, was judged too fragile to bet the demo on. So the Python code no longer ships: it is the verified specification, and its exact logic was ported natively, leaving one running application. What the product does did not change, only what it is built from.

Change 2: classifier chosen by measurement. Benchmarks (Section II-A) show base Laya is useless zero-shot (9.3%) and fine-tuning makes it work (87.6%). The LNN is 11$\times$ faster and 95$\times$ smaller but 30 points less accurate and almost never detects form fields, so fine-tuned Laya on the NPU is the recommended classifier. The 0.8 confidence gate is now backed by measured results.

Table 1: Classifier benchmark, 259 test turns.

Model (build)	Type acc.	Relev. acc.	Field F1	ms/ turn	File MB
Base Laya (W8A16, NPU)	9.3%	—	—	10.8	859
Fine-tuned Laya (fp32, CPU)	88.0%	95.5%	0.659	510	1507
Fine-tuned Laya (W8A16, NPU)	**87.6%**	95.2%	0.661	**10.9**	860
Fine-tuned Laya (W8A8, NPU)	1.9%	59.1%	0.014	6.8	759
LNN (W8A8, NPU)	57.5%	74.7%	0.047	0.96	9

Table 2: Logic experiments and outcomes.

Experiment	Result
Routing / form-filling / clarifying state machine (Python, scripted stub), 8 scenarios	8 of 8 passed by inspection of printed output; 2 real bugs fixed
Review, correction, idempotent submit, PBKDF2 admin auth, department-scoped access, versioning and rollback (Python)	Each verified against real executed output; 3 issues found and fixed
GNN-style relatedness and DynoTears-style causal proxy (simplified, background)	Sparse data gives 0 edges; engineered data isolates the true signal; 1 statistical bug fixed
Kotlin port: all scenarios as asserting JUnit tests, plus on-device database tests (emulator, API 37)	17 JVM and 57 instrumented tests, 0 failures
Password hashing across runtimes	Byte-for-byte match with Python on desktop JVM and Android, including non-ASCII input
Mutation tests on the 0.8/0.15 rule and the 6-turn bound	Suite caught each deliberate break; code restored

II. Experiments Run So Far and Results

A. Classifier benchmark on the kit (Snapdragon 8 Elite, Hexagon v79 NPU, QAIRT/SNPE 2.47). Data: 1,048 / 116 / 128 train / validation / test conversations (259 held-out test turns), split by conversation; 30 labels (28 types plus undetermined and none; chance 3.3%). Table 1 gives the key results.

- **Fine-tuning is essential.** Base embeddings are nearly identical for every input (cosine 0.90). LoRA on 7.3M of 395M parameters took type accuracy from 11.6% to 88.0% (fp32), with 95% relevance accuracy and 0.66 field F1.
- **NPU is lossless at W8A16 and 47$\times$ faster:** 87.6% vs 88.0% on CPU, 10.9 vs 510 ms. Fully 8-bit activations collapse Laya to chance (activation outliers); the LNN quantizes cleanly even at W8A8.
- **Bugs fixed.** The attention-mask value -3.4×10^{38} destroyed quantized attention (cosine 0.14); exporting -60 fixed it (0.985) and cut latency from 18.8 to 10.8 ms. SNPE wrote only the last of three output tensors, so the heads are concatenated into one.
- **Department-first classification never helps** (87.3% vs 87.6% flat): a flat head already ranks departments. **Shuffle-and-vote hurts** (86.7% vs 90.6% final-turn): out-of-order inputs are out of distribution and the views err together.
- **Confidence gate works.** On the final decision: always answer 90.6%; answer at $\geq$0.8 else confirm top-2, 94.5%; else offer top-3, **96.1%**. 73.4% of conversations are answered directly (96.8% correct); about 1 in 4 get one extra tap. The right type is in the top-3 for 97.7%.
- **Error sources.** 11 of 32 wrong turns are "undetermined vs a type" (when to commit, not which type); main confusions are lab equipment vs printer/device, lighting vs conference-room AV, theft vs suspicious person.

B. Logic experiments. Table 2 lists the other experiments; all results come from executed test output.

State machine scenarios. Each targeted a specific failure: (1) a confident single match skips disambiguation; (2) two close candidates trigger a disambiguating question instead of a guess; (3) zero candidates give a generic clarifying question, not a stall; (4) two incidents in one narration split into two independently evaluated sessions; (5)

a field answered once is never re-asked, even across a disambiguation round-trip; (6) "I don't know" is stored as a final answer; (7) FORM_FILLING forces progress after 6 turns; (8) CLARIFYING stops the moment the user is done.

Review and submit coverage. The review summary always shows every required field with its status, and optional fields only once answered. Correction creates a tracked revision and walks a dependency map (description → priority, severity → priority). Cancel leaves zero trace in the incidents table. Submit is idempotent, so a duplicate confirmation can never create two rows. Zone resolution is tiered: a logged function-call result, then a name match, then left unresolved rather than blocking submission. A department admin sees only their department's queue; a super admin sees the whole organization.

Bugs the experiments caught.

- *Ambiguity rule.* The confidence rule treated "several plausible labels for one incident" like "two incidents," so an ambiguous single incident could spawn a second, empty draft. Fixed with an explicit "distinct incidents mentioned" signal checked before any confidence math.
- *Contract violation.* A later fix added a dialogue-act value ("stop_clarifying") outside the frozen set of five. It was replaced with a dedicated boolean, keeping the contract intact.
- *Fabricated test row.* A harness invented a fake conversation-turn row to satisfy a foreign key. We required written confirmation that the integrated flow only ever uses the genuine turn from the user's spoken correction.
- *Schema gap.* `incidents.zone_id` was NOT NULL, impossible to satisfy once "I don't know" or a timeout is allowed for Location. It is now nullable; the zone is resolved opportunistically and never blocks submission.
- *Version activation.* Reading the SQL confirmed that activating a version deactivates every sibling, not only the latest.
- *Causal heuristic gate.* The minimum-sample check counted all incidents touching a factor, not the exact factor-to-effect pairs, so 5 incidents produced 8 false "causal" edges. After the fix, 0; a factor present on every incident is correctly excluded.
- *Print-only tests.* The Python tests never asserted pass or fail. The Kotlin suite asserts every scenario and is now authoritative.

C. What the Kotlin port involved.

- The Conversation Engine contract was fixed to the frozen design: closed dialogue-act and classification-outcome enums that reject unknown values instead of coercing them.
- The full 16-table schema runs on Android SQLite with foreign keys explicitly enabled and checked on a device (Android leaves them off by default).
- Reading the real taxonomy showed 28 incident types and 201 fields, no priority or severity field (priority is each type's fixed default level), and location as one `space_id` room name, not building plus floor. A single *FieldRole* mapping layer now supplies every field name to the logic.
- Demo zones were seeded from the taxonomy's own rooms (Conference Room 2, Pantry, Cafeteria, Men's Washroom, Server Room, RF Lab, Parking) so a spoken location resolves to a real pin.
- An admin status workflow (Open → In Progress → Resolved) is department-scoped and protected against two admins updating the same incident at once.

The optional pieces are deliberately simple and labelled as such: a weighted similarity score (shared zone, type or department, time, keywords) for related incidents, and a lagged co-occurrence/lift statistic for candidate causal factors. Neither is a trained GNN or real DynoTears; both run in the background and cannot fail a submission.

III. Progress on Testing

Working. The end-to-end system runs: a back-and-forth conversation through the app is feasible, and it has successfully produced a submitted incident. Routing, review, correction, submission, authentication, department-scoped access, versioning and rollback, taxonomy and zone seeding, and status transitions are wired into real, hand-driven app screens and pass 17 JVM and 57 instrumented tests.

Verified at platform level. Foreign-key enforcement is on for both read and write connections on a real Android database (live PRAGMA query). Password hashing matches the reference implementation byte for byte. On the kit, fine-tuned Laya on the NPU matches the Colab GPU result exactly in fp32 (88.0%).

Open gaps, by importance.

1. **Test data is synthetic.** The benchmark set is LLM-written like the training data, so accuracy is optimistic, and 259 turns means differences under 3–4 points are noise. A team-written, spoken test set is needed. The 0.8 threshold was read off test results, which flatters it; it must be chosen on validation.
2. **Full-loop latency not yet measured.** Classifier timing excludes tokenization and app overhead; ASR, LLM and TTS are not yet benchmarked against the targets (listening $<$100 ms; interrupt $<$150 ms; fast-path follow-up $<$1 s; semantic-path $<$2 s). Memory, power and thermal data are missing; Laya needs about 380 MB of weights.
3. **Live map in progress.** The queue and status controls work; zone highlights and pins do not yet render.
4. **Two screens not built** by choice: department-admin creation and draft recovery (tested backend exists).

IV. Plan for the Remaining Time

In the order it will be tackled, and why:

1. **Finish the live map** (zone highlights, pins, tap-to-open dossier): a stated differentiator, and the seeded zones and resolved locations it needs already exist.
2. **Push everything to `main`** in logical commits, including the Python specification and benchmark scripts. An M2 requirement, and it closes the risk of verified work sitting uncommitted.
3. **Measure the full loop on the kit:** ASR, classifier, LLM and TTS latency (P95) against the targets, plus memory, power and thermal behavior. This cannot be simulated.
4. **Raise classifier accuracy:** choose the gate threshold on validation; ask "A, B or C?" on low confidence and commit only at submit; add data for the confusable pairs; fine-tune the 6 MB Ettin-17m with the same notebook to see how much accuracy a $\sim$60$\times$ smaller model keeps.
5. **Spoken test set:** a short team-recorded set to replace synthetic numbers.
6. **If time remains:** the department-admin creation and draft-recovery screens; the GNN/DynoTears prototype is done and only polished if time allows.
7. **Rehearse the judge walkthrough** with the network disabled: an English run with an "I don't know" and a clarifying question; a Hindi or code-mixed run with a disambiguation and a barge-in; a REVIEW correction; the admin queue and map updating live. It depends on steps 1–4 and is where rough edges will surface.